

Secondary Indexes: GSI e LSI

Consultas Flexíveis com Índices Secundários no DynamoDB

AWS Certified Developer Associate

Instrutor: Marlon Anesi

O que são Secondary Indexes?

Sem Indexes: Problema

- PK = orderID: query eficiente apenas por orderID
- Buscar pedidos por customerID?
- Sem index = SCAN na tabela inteira
- Resultado: lento, caro, risco de throttling



Com Index: Solução

- Index = visão alternativa da tabela
- Permite queries por atributos não-chave
- DynamoDB mantém o index automaticamente
- Queries rápidas em múltiplos atributos

Dois tipos de Secondary Indexes no DynamoDB:

GSI – Global Secondary Index

- PK e SK completamente diferentes da tabela base
- Atravessa TODAS as partições (Global)
- Criado a qualquer momento
- Capacidade própria (RCU/WCU dedicados)

LSI – Local Secondary Index

- Mesma PK da tabela base, SK diferente
- Confinado à mesma partição (Local)
- Criado SOMENTE ao criar a tabela
- Compartilha capacidade com a tabela base

GSI – Global Secondary Index em Detalhe

Características do GSI

- PK e SK podem ser QUALQUER atributo da tabela
- Valores não precisam ser únicos no index
- Máximo de 20 GSIs por tabela
- Consistência eventual (eventually consistent)
- Projeção: ALL, KEYS_ONLY ou INCLUDE
- Capacidade dedicada independente da tabela
- Pode ser criado e deletado a qualquer momento

Exemplo Prático

Tabela base: Orders

PK: orderID	date	customerID
ORD-001	2024-01-15	CUST-123
ORD-002	2024-01-16	CUST-456

GSI: CustomerOrders-Index

PK: customerID | SK: date

PK: customerID	SK: date	orderID
CUST-123	2024-01-15	ORD-001
CUST-456	2024-01-16	ORD-002

Query eficiente: todos os pedidos do CUST-123

LSI – Local Secondary Index em Detalhe

Características do LSI

- Mesma PK da tabela base (obrigatório)
- Sort Key diferente da tabela base
- Máximo de 5 LSIs por tabela
- Suporta leitura strongly consistent
- Projeção: ALL, KEYS_ONLY ou INCLUDE
- Compartilha RCU/WCU com a tabela base
- ATENCAO: criado SOMENTE ao criar a tabela!

Exemplo Prático

Tabela: UserPosts (PK=userID, SK=timestamp)

PK: userID	SK: timestamp	likes
USER-A	2024-01-10	150
USER-A	2024-01-12	320

LSI: UserPostsByLikes-Index

Mesma PK: userID | Nova SK: likes

PK: userID	SK: likes	timestamp
USER-A	320	2024-01-12
USER-A	150	2024-01-10

Resultado: posts do USER-A ordenados por likes!

GSI vs LSI: Comparação Direta

Característica	GSI	LSI
Partition Key	Qualquer atributo	MESMA da tabela base
Sort Key	Qualquer atributo	Atributo diferente
Quando criar	A qualquer momento	Só ao criar a tabela
Consistência	Eventual apenas	Eventual ou Strong
Capacidade	Dedicada (própria)	Compartilhada
Limite por tabela	Até 20 GSIs	Até 5 LSIs
Escopo de query	Todas as partições	Mesma partição

Quando Usar GSI vs LSI?

Use GSI quando...

- Precisar buscar por PK diferente da tabela
- Quiser criar o index numa tabela já existente
- Precisar de consultas globais entre partições
- Precisar de capacidade dedicada para o index
- Exemplo: buscar pedidos por customerID numa tabela com PK=orderID

Use LSI quando...

- Precisar ordenar pela mesma PK com SK diferente
- Precisar de leitura strongly consistent no index
- A tabela ainda não foi criada (design inicial)
- Consultas ficam dentro da mesma partição
- Exemplo: posts do userID ordenados por likes ao invés de timestamp

Pontos-chave para Revisão

1. GSI – Global Secondary Index

- PK e SK diferentes | até 20 por tabela | consistência eventual
- Capacidade própria | criado a qualquer momento (total flexibilidade)

2. LSI – Local Secondary Index

- Mesma PK | SK diferente | até 5 por tabela | strongly consistent possível
- CRIADO SOMENTE ao criar a tabela – não pode ser adicionado depois!

3. Projeção de Atributos

- ALL: replica todos os atributos (mais custo, sem fetch extra)
- KEYS_ONLY: apenas chaves (menos custo, pode precisar fetch extra)
- INCLUDE: atributos específicos (equilíbrio custo x performance)

LEMBRE-SE: Se a tabela já existe e precisa de novo index > use SEMPRE GSI! LSI só na criação da tabela!